Date: 14th - July - 2026

RNN, LSTM & GRU

Name: Zulekha D/O Muhammad Usman

- **RNN (Recurrent Neural Network):**

A type of neural network that focuses on classification (sequence -> label) or predicting the next word in a sentence (sequence -> sequence), it works well for small projects and learns better with time if programmed well, where it keeps feeding its previous output to the next step and those outputs are kept in a memory like hidden layer called "short memory".

Unfortunately it has one major issue that it fails after certain amount of data which is called "vanishing/exploding gradient problem" because the numbers shrink so close to zero (**vanishing gradient**) or get multiplied to huge numbers (**exploding gradients**), where it starts forgetting the previous sequences making it hard to relate back and make sense, it's like alzheimer in humans, happens because of the mathematical **chain rule** on which RNN is based whereas it lacks a proper memory to save those huge numbers.

How it works: it keeps updating the short term memory in its hidden state with each sequence where current memory = past memory + new input, based on this formula:

$$h_t = \tanh(W_{hh}\, h_{t-1} + W_{xh}\, x_t + b_h)$$

Where:

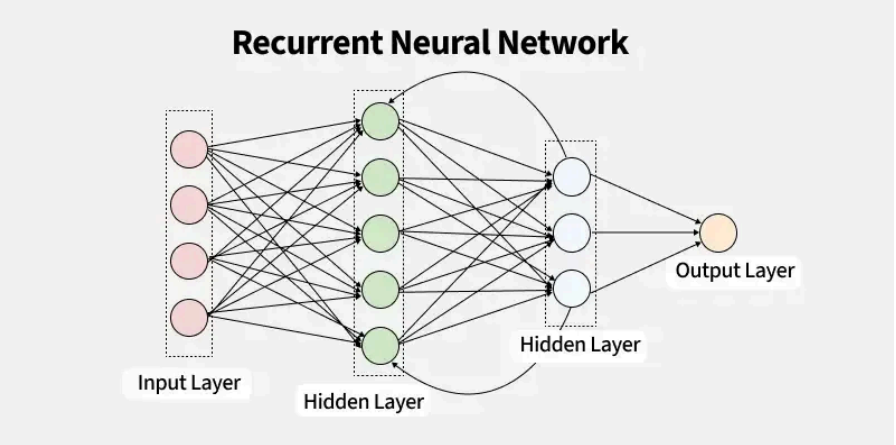
- W_{hh} : Weight matrix multiplied by the **previous hidden state** (h_{t-1}).
- W_{xh} : Weight matrix multiplied by the **current input** (x_t).
- b_h : The bias vector for the hidden layer.
- $\tanh$: The activation function (squashes values between -1 and 1) to keep the hidden state values stable.

whereas the output is renewed based on weights and biases:

$$y_t = \text{activation}(W_{hy}\, h_t + b_y)$$

Where:

- W_{hy} : Weight matrix applied to the current hidden state to produce the output.
- b_y : The bias vector for the output layer.
- The activation is typically **Softmax** (for predicting class probabilities, like the next word) or **Linear** (for numerical regressions, like stock price prediction).



- **LSTM (Long Short Term Memory):**

  To solve the gradient descent problem in RNN the LSTM was introduced, it works like an **internal memory loop** that works alongside short memory like a conveyer belt that allows it to hold gradients and data without shrinking, where it has 3 gates working like filters to regulate it (forget gate, input gate, output gate)

  - ☐ **Forget gate:**

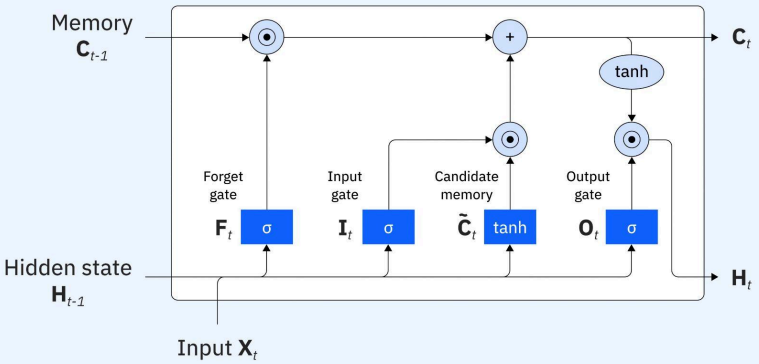
    Decides what the memory keeps and what it **cuts off**

  - ☐ **Input gate:**

    mainly about "**information scaling**", along with the processes of the network i decides which of the new introduced data are worth writing into the long term memory

  - ☐ **Output gate:**

    Decides what the **next step** would be based on the running process



- **GRU (Gated Recurrent Unit):**

  The same function of LSTM, we can consider it a newer version of LSTM since it trains faster because it has fewer parameters, it has to deal with two gates instead of three, i.e. Update gate and Reset gate

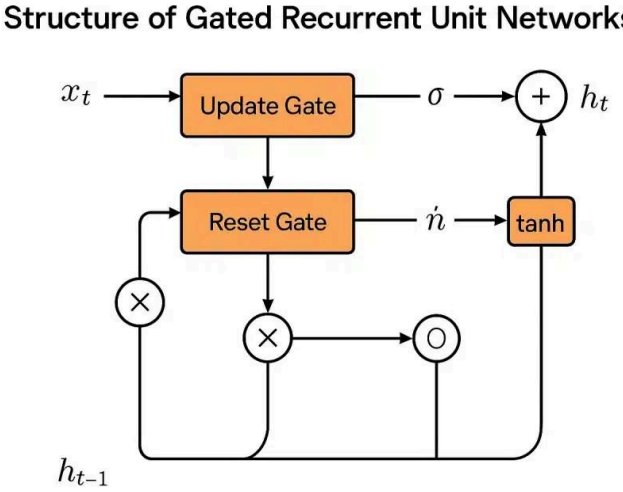
  - ☐ **Update gate:**

    combines both **Forget and input gate** from LSTM, where it decides both how much of the data will be used/dropped and also work on it

simultaneously by bringing in new data and sending it to the long term memory

- [ ] **Reset gate:**

  Decides how much of the **past state** data to forget



Model	RNN	LSTM	GRU
Gating	none	3 gates: (forget, input, output)	2 gates: (update, reset)
Memory span	short	long	long
Speed:	fastest	slowest	moderate
When to use	for ultra-simple, short-sequence tasks where computational resources are highly limited.	when you have complex, long sequences (like long text paragraphs or complex time-series) where tracking long-term dependencies is absolutely critical.	when you want a great balance between performance and speed, or when training on smaller datasets where LSTMs might overfit.